

Beginner to Advanced

Git and GitHub Practical Handbook

Commands, workflows, examples, challenges, and interview questions

Prepared for Balaji Baskar
MEAN Stack and DevOps Learning Path

Edition: September 2026

How to Use This Handbook

This handbook is designed for hands-on learning. For every topic, read the concept, run the commands in a practice repository, observe the result, and complete the mini challenge.

Recommended practice: Create one folder named `git-lab`. Use a separate subfolder for each chapter so that experiments do not affect one another.

Learning Roadmap

1. Git and version control fundamentals
2. Installation and configuration
3. Repository lifecycle and commits
4. Inspecting changes and history
5. Ignoring files
6. Branching and merging
7. Merge conflicts
8. GitHub repositories and remotes
9. Push, pull, fetch, and clone
10. Pull requests and code review
11. Stash
12. Restore, reset, and revert
13. Tags and releases
14. Cherry-pick
15. Rebase and interactive rebase
16. GitHub Issues and Projects
17. GitHub Actions CI/CD
18. Team workflows and best practices
19. Troubleshooting and interview revision

Command Safety Legend

Level	Meaning
Safe	Normally preserves committed history.
Use carefully	Can discard local work or rewrite history.
Shared branch warning	Avoid history rewriting after commits are shared.

1. Git and Version Control Fundamentals

What is Git?

Git is a distributed version control system. It records snapshots of a project, lets you compare versions, create parallel lines of work, and collaborate without manually copying folders.

Git vs GitHub

Git	GitHub
Runs locally.	Hosts Git repositories online.
Tracks versions and branches.	Adds collaboration, pull requests, issues, security features, and automation.
Works without internet.	Internet is normally required to synchronize.

Working files -> Staging area -> Local repository -> Remote repository

Practical example

```
mkdir git-basics
cd git-basics
git init
echo "Hello Git" > app.txt
git status
```

Observe: `app.txt` appears as an untracked file because Git sees it but has not added it to a snapshot.

Mini challenge

1. Create `index.html`.
2. Run `git status`.
3. Predict what Git will report before reading the output.

2. Installation and Configuration

First, verify that Git is available and configure the identity stored in your commits.

```
git --version
git config --global user.name "Balaji Baskar"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main
git config --list
```

Global vs local configuration

Scope	Command	Use
Global	git config --global ...	Default for all repositories.
Local	git config ...	Overrides settings only in the current repository.

Example: separate work identity

```
cd company-project
git config user.name "Balaji Baskar"
git config user.email "balaji@company.example"
git config --local --list
```

Important: Use the email connected to the relevant Git hosting account if you want commits attributed correctly.

3. Repository Lifecycle and Commits

Edit -> git add -> git commit -> repeat

Create a repository

```
mkdir employee-api
cd employee-api
git init
echo "console.log('API started');" > server.js
git status
```

Stage and commit

```
git add server.js
git status
git commit -m "Initialize employee API"
```

Stage choices

```
git add server.js      # one file
git add src/           # one folder
git add .              # all current changes
git add -p             # review changes in sections
```

Good commit messages

Good	Avoid
Add JWT validation middleware	changes
Fix expired-session status update	fixed
Refactor employee search service	code update

Rule: Keep each commit focused on one logical change. A focused commit is easier to review, revert, and cherry-pick.

4. Inspect Changes and History

Status and differences

```
git status
git diff           # unstaged changes
git diff --staged  # staged changes
git diff HEAD      # all changes since last commit
```

Example

```
echo "console.log('Database connected');" >> server.js
git diff
git add server.js
git diff --staged
```

History commands

```
git log
git log --oneline
git log --oneline --graph --decorate --all
git show HEAD
git show <commit-id>
```

Practical habit: Run `git diff` before staging and `git diff --staged` before committing. This catches accidental debug code and unrelated changes.

5. Ignore Files with .gitignore

A .gitignore file defines intentionally untracked files that Git should ignore.

MEAN stack example

```
# Dependencies
node_modules/

# Build output
dist/
coverage/

# Environment and secrets
.env
.env.*

# Logs
*.log
logs/

# Editor and OS
.vscode/
.DS_Store
```

Important behavior

If a file is already tracked, adding it to .gitignore does not stop tracking it.

```
git rm --cached .env
git commit -m "Stop tracking environment file"
```

Security: Never commit tokens, passwords, private keys, or production connection strings. If a secret was committed, removing the file later does not remove the secret from earlier history. Rotate the secret.

6. Branching

A branch is an independent line of development. Use branches to isolate features, fixes, experiments, and releases.

Core commands

```
git branch
git switch -c feature/login
git switch main
git branch -d feature/login
```

MEAN stack example

```
git switch -c feature/jwt-auth
# Edit auth.service.ts and backend middleware
git add .
git commit -m "Add JWT authentication flow"
git switch main
```

Naming examples

```
feature/login
feature/employee-search
bugfix/session-expiry
hotfix/production-certificate
docs/api-setup
```

Remember: Creating a branch does not automatically switch to it unless you use `git switch -c` or `git checkout -b`.

7. Merge

`git merge` combines another branch into the branch you currently have checked out.

```
git switch main
git merge feature/login
```

Meaning: Take the commits reachable from `feature/login` and integrate them into `main`.

Fast-forward merge

If `main` has not moved since the feature branch was created, Git can move the branch pointer forward.

Merge commit

If both branches contain new commits, Git may create a merge commit that joins the histories.

Useful options

```
git merge --no-ff feature/login # force merge commit
git merge --abort                # stop a conflicted merge
git log --oneline --graph --all
```

Live exercise

```
git switch -c feature/dashboard
echo "Dashboard" > dashboard.txt
git add .
git commit -m "Add dashboard placeholder"
git switch main
git merge feature/dashboard
```

8. Merge Conflicts

A conflict occurs when Git cannot safely decide how to combine changes, commonly when both branches modify the same lines.

Conflict markers

```
<<<<<<< HEAD
const appName = "HR Portal";
=====
const appName = "Employee Portal";
>>>>>> feature/branding
```

Resolution workflow

1. Run `git status` to identify conflicted files.
2. Open each file and decide the final correct content.
3. Remove the conflict markers.
4. Test the application.
5. Stage the resolved files and commit.

```
git status
# Edit the conflicted file
git add src/app/app.component.ts
git commit -m "Resolve branding merge conflict"
```

Abort if necessary

```
git merge --abort
```

Reduce conflicts: Pull or fetch regularly, keep branches short-lived, make small PRs, and coordinate when multiple developers edit the same module.

9. GitHub Repository and Remotes

A remote is a named connection to another repository. The conventional default name is `origin`.

Connect an existing local project

```
git remote add origin https://github.com/USERNAME/employee-portal.git
git remote -v
git push -u origin main
```

Change or remove a remote

```
git remote set-url origin NEW_REPOSITORY_URL
git remote remove origin
```

Authentication

For HTTPS, use an approved credential flow or personal access token where required. SSH is another common option after adding your public SSH key to the hosting account.

```
ssh-keygen -t ed25519 -C "your-email@example.com"
git remote set-url origin git@github.com:USERNAME/employee-portal.git
```

Do not share: A private SSH key must remain private. Only the public key is uploaded to a service.

10. Clone, Fetch, Pull, and Push

Command	Purpose
<code>git clone URL</code>	Create a local copy of a remote repository.
<code>git fetch</code>	Download remote references without integrating them.
<code>git pull</code>	Fetch and then integrate changes.
<code>git push</code>	Upload local commits.

Examples

```
git clone https://github.com/USERNAME/employee-portal.git
cd employee-portal
git fetch origin
git log --oneline main..origin/main
git pull origin main
git push origin main
```

Push a new feature branch

```
git switch -c feature/profile
git add .
git commit -m "Add employee profile page"
git push -u origin feature/profile
```

-u: Sets the upstream branch so later you can normally run only `git push` and `git pull`.

11. Pull Requests and Code Review

A pull request proposes merging one branch into another and provides a place to review changes, discuss issues, run checks, and approve the update.

Developer workflow

```
git switch main
git pull origin main
git switch -c feature/login-validation
# Make and test changes
git add .
git commit -m "Validate login form inputs"
git push -u origin feature/login-validation
```

Example PR description

```
Title: Add login form validation

Summary:
- Validate required username and password
- Show user-friendly validation messages
- Add unit tests for invalid input

Testing:
- npm test
- Manual login checks
```

Review update

```
# Fix reviewer comments
git add .
git commit -m "Handle empty username validation"
git push
```

The existing pull request updates automatically when new commits are pushed to its branch.

Good PR: Small scope, clear description, test evidence, no unrelated formatting changes, and no secrets.

12. Git Stash

Stash temporarily stores uncommitted work, allowing you to switch context without making an unfinished commit.

Production-fix scenario

```
# On feature/login with unfinished changes
git stash push -m "WIP login validation"
git switch main
git pull origin main
git switch -c hotfix/session-expiry
# Fix and commit

git switch feature/login
git stash pop
```

Stash commands

```
git stash list
git stash show -p stash@{0}
git stash apply stash@{0} # apply but keep stash
git stash pop             # apply and remove on success
git stash drop stash@{0}
git stash clear
```

Include untracked files

```
git stash -u
```

Be careful: Stash is temporary working storage, not a replacement for meaningful commits or remote backup.

13. Restore, Reset, and Revert

Command	Typical purpose	Risk
<code>git restore</code>	Discard or unstage file changes.	Can discard uncommitted work.
<code>git reset</code>	Move HEAD/branch and optionally update index/worktree.	Can rewrite history or delete work.
<code>git revert</code>	Create a new commit that reverses an earlier commit.	Usually safest for shared history.

Restore examples

```
git restore server.js
git restore --staged server.js
```

Reset examples

```
git reset --soft HEAD~1  # remove commit, keep staged
git reset HEAD~1         # remove commit, keep unstaged
git reset --hard HEAD~1  # remove commit and changes
```

Revert examples

```
git revert HEAD
git revert <commit-id>
```

Decision rule: Unpushed local commit: reset may be suitable. Shared or pushed commit: prefer revert.
Uncommitted file change: restore may be suitable.

14. Tags and Releases

Tags mark important commits, commonly software versions.

Annotated tag

```
git tag -a v1.0.0 -m "First production release"
git show v1.0.0
git push origin v1.0.0
```

List and delete

```
git tag
git tag -d v1.0.0
git push origin --delete v1.0.0
```

Version example

Change	Example
Breaking change	2.0.0
Backward-compatible feature	1.1.0
Backward-compatible fix	1.0.1

Practical use: Tag the exact commit deployed to production. This makes rollback and release comparison easier.

15. Cherry-Pick

Cherry-pick applies the change introduced by one or more specific commits onto the current branch.

JWT hotfix example

```
# Find the desired commit
git log --oneline feature/login

# Apply it to the release branch
git switch release/1.0
git cherry-pick 3f8a21c
```

If a conflict occurs

```
# Resolve files, then
git add .
git cherry-pick --continue

# Or cancel
git cherry-pick --abort
```

Use when: You need an isolated bug fix without merging all unfinished work from the source branch.

Avoid overuse: Repeated cherry-picking can duplicate commits and make branch relationships harder to understand.

16. Rebase and Interactive Rebase

Rebase reapplies commits on top of another base commit, often producing a linear history.

Update a feature branch

```
git fetch origin
git switch feature/login
git rebase origin/main
```

Resolve rebase conflict

```
# Edit files
git add .
git rebase --continue

# Skip current commit if appropriate
git rebase --skip

# Cancel
git rebase --abort
```

Interactive rebase

```
git rebase -i HEAD~4
```

Common actions include pick, reword, squash, fixup, and drop.

```
pick a1b2c3 Add login form
fixup d4e5f6 Fix typo
squash a7b8c9 Add validation
reword d1e2f3 Add tests
```

Shared branch warning: Rebase rewrites commit IDs. Avoid rebasing commits that teammates already depend on unless the team workflow explicitly allows it. If pushing a rebased personal branch, prefer `git push --force-with-lease` over `--force`.

17. GitHub Issues and Projects

Issue example

Title: Session status not updated after JWT expiry

Expected:

The user should be redirected to login and session state cleared.

Actual:

The API token expires, but the UI still shows an active session.

Steps:

1. Log in.
2. Wait for token expiry.
3. Call a protected API.

Acceptance criteria:

- Clear local session state on 401.
- Redirect to login.
- Add a regression test.

Link work to the issue

```
git switch -c bugfix/123-session-expiry
git commit -m "Fix session handling after token expiry"
git push -u origin bugfix/123-session-expiry
```

In the pull request description, reference the issue according to your repository's conventions. Projects can organize issues and pull requests into views such as backlog, in progress, review, and done.

18. GitHub Actions: CI/CD Fundamentals

A workflow is a YAML file stored under `.github/workflows/`. It can run jobs in response to repository events.

Node.js CI example

```
name: Node CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build
        run: npm run build
```

Use secrets

```
env:
  JWT_SECRET: ${ secrets.JWT_SECRET }
  MONGO_URI: ${ secrets.MONGO_URI }
```

Never print secrets: Do not echo secrets to workflow logs. Use least-privilege credentials and environment protection for deployment.

19. MEAN Stack CI Example

For a repository with separate `client` and `server` folders, run the jobs independently.

```
name: MEAN Stack CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  client:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: client
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: client/package-lock.json
      - run: npm ci
      - run: npm test -- --watch=false
      - run: npm run build

  server:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: server
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: server/package-lock.json
      - run: npm ci
      - run: npm test
```

Practical extension: Add linting, dependency scanning, artifact upload, and deployment only after tests pass.

20. Team Workflows

Feature branch workflow

```
main -> feature branch -> commits -> push -> pull request -> review -> merge
```

```
git switch main
git pull origin main
git switch -c feature/employee-search
# Work, test, commit
git push -u origin feature/employee-search
# Open PR and address review comments
```

Git Flow model

```
main
develop
  feature/*
  release/*
  hotfix/*
```

Git Flow is structured but can be heavy. Teams with frequent delivery may prefer trunk-based development with short-lived branches and strong automated testing.

Protected branch practices

- Require pull requests.
- Require successful checks.
- Require approvals.
- Prevent direct force pushes.
- Limit who can bypass protections.

21. Everyday Professional Workflow

Start work

```
git switch main
git pull origin main
git switch -c feature/report-filter
```

During work

```
git status
git diff
git add -p
git commit -m "Add report date filter"
```

Before creating PR

```
git fetch origin
git rebase origin/main
npm test
npm run build
git push -u origin feature/report-filter
```

After merge

```
git switch main
git pull origin main
git branch -d feature/report-filter
git fetch --prune
```

Quality checklist: Focused changes, meaningful commits, tests pass, build passes, no secrets, no generated files unless required, and clear PR notes.

22. Troubleshooting Guide

Remote origin already exists

```
git remote -v
git remote set-url origin NEW_URL
```

Push rejected because remote is ahead

```
git fetch origin
git rebase origin/main
# Resolve if required
git push
```

Wrong branch commit

```
# Create the intended branch at current commit
git branch feature/correct-branch
# Then move the original branch carefully as appropriate
```

Detached HEAD

```
git switch -c recovery/my-work
```

Find a lost commit

```
git reflog
git show <commit-id>
git switch -c recovery/lost-work <commit-id>
```

Remove stale remote branches

```
git fetch --prune
```

Before destructive recovery: Create a backup branch: `git branch backup-before-fix`.

23. Interview Revision

Question	Short answer
Git vs GitHub?	Git tracks versions; GitHub hosts repositories and collaboration workflows.
What is staging?	The selected changes prepared for the next commit.
Merge vs rebase?	Merge joins histories; rebase reapplies commits onto a new base and rewrites IDs.
Fetch vs pull?	Fetch downloads references; pull fetches and integrates.
Reset vs revert?	Reset moves history; revert creates a new undo commit.
What is cherry-pick?	Apply a specific commit's change to the current branch.
What is stash?	Temporary storage for uncommitted working changes.
What is a PR?	A reviewed proposal to merge changes between branches.
What is CI?	Automated integration checks such as install, test, lint, and build.

24. Final Hands-On Project

Project: Employee Portal Repository

1. Create a GitHub repository.
2. Create client and server applications.
3. Add a secure .gitignore.
4. Use main plus feature branches.
5. Create separate PRs for login, employee CRUD, and session expiry handling.
6. Intentionally create and resolve one merge conflict.
7. Use stash while switching to a hotfix.
8. Revert one test commit safely.
9. Cherry-pick one bug fix to a release branch.
10. Add a CI workflow for tests and builds.
11. Create a release tag.

Completion evidence

- Clean graph from `git log --oneline --graph --all`.
- Closed issues linked to PRs.
- Passing GitHub Actions checks.
- No secrets in the repository.
- README includes setup, test, and build commands.

Career outcome: This project demonstrates practical Git collaboration and CI skills alongside your MEAN stack development work.

25. Command Cheat Sheet

Setup and status

```
git init
git clone URL
git status
git config --list
```

Undo

```
git restore FILE
git restore --staged FILE
git revert COMMIT
git reset --soft HEAD~1
```

Stage and commit

```
git add .
git add -p
git commit -m "Message"
git diff --staged
```

Advanced

```
git stash
git cherry-pick COMMIT
git rebase main
git rebase -i HEAD~3
```

Branches

```
git branch
git switch -c NAME
git switch main
git merge NAME
```

Recovery

```
git reflog
git merge --abort
git rebase --abort
git cherry-pick --abort
```

Remote

```
git remote -v
git fetch origin
git pull
git push
```

History

```
git log --oneline --graph --all
git show HEAD
git tag -a v1.0.0 -m "Release"
```

Practice commands in a test repository before using destructive operations in a real project.